SHOPEASE

A Full Stack E-Commerce Web Application

Subject :Web Application Development

Project Report

Technology: React.js + Node.js + Express.js + MongoDB

Sr.No	Group Member Name	Enrollment No.
1	Vasava Vinit	240280116149
2	Vasava Meet	240280116148
3	Vasava Vatsal	240280116151

Details	Information
Semester	5B div
College/University	L.D College of Engineering
Academic Year	2025-2026

1. Abstract

ShopEase is a full-stack e-commerce web application developed using React.js for the frontend, Node.js and Express.js for the backend, and MongoDB with Mongoose for data storage. The application provides customer features such as registration, login, product browsing, product details, cart, wishlist, checkout and order tracking. It also provides administrative features for managing products and orders. REST APIs connect the React client with the backend, while JWT-based authentication is used to protect user and administrator operations.

2. Introduction

E-commerce applications allow users to discover products, add items to a cart, place orders and track purchases through a digital platform. ShopEase was developed as a practical full-stack web application to demonstrate frontend development, backend API development, database integration, authentication and CRUD-based administration in one project.

3. Problem Statement

Traditional small-scale product selling systems may not provide an integrated platform for product browsing, customer accounts, cart management and order administration. ShopEase addresses this problem by providing a centralized web application where customers can purchase products and administrators can manage the product catalogue and orders.

4. Objectives

- Develop a responsive and user-friendly e-commerce interface.
- Implement secure registration and login using JWT authentication.
- Store users, products and orders in MongoDB.
- Provide product browsing, details, cart and wishlist functionality.
- Implement checkout and order creation.
- Provide an administrator dashboard for product and order management.
- Demonstrate REST API communication between React and Node.js.

5. Technologies Used

Technology	Purpose	Use in Project
React.js	Frontend UI	Pages, components and routing
Node.js	Runtime	Backend execution
Express.js	Web framework	REST APIs and server routes
MongoDB	Database	Persistent application data
Mongoose	ODM	MongoDB schemas and queries
JWT	Authentication	Login/session authorization
Bcryptjs	Password security	Password hashing
Axios	HTTP client	Frontend API calls
CSS	UI styling	Layout and visual design
Postman	API testing	Testing REST endpoints

6. System Architecture

The application follows a three-layer full-stack architecture:

React.js Frontend → REST API → Node.js/Express Backend → Mongoose → MongoDB

The frontend sends HTTP requests to backend endpoints. Express routes validate and process requests, Mongoose communicates with MongoDB, and JSON responses are returned to the React application.

7. Main Modules

Module	Description
User Authentication	Registration, login, JWT authentication and logout.
Product Catalogue	Display products, categories, prices, stock and product details.
Cart	Add products, change quantities, remove items and calculate totals.
Wishlist	Save and manage preferred products.
Checkout	Collect customer and delivery information and create an order.
Orders	Store customer orders and display order status.
Admin Dashboard	Manage products and monitor/update orders.

8. Database Design

The project uses MongoDB. Important collections/models include:

- Users: name, email, password, role and wishlist.
- Products: name, price, category, image, description and stock.
- Orders: user, items, customer details, payment method, total and status.

9. API Design

Method	Endpoint	Purpose
POST	/api/auth/register	Create user account
POST	/api/auth/login	Authenticate user
GET	/api/products	Get all products
GET	/api/products/:id	Get one product
POST	/api/products	Add product (admin)
PUT	/api/products/:id	Update product (admin)
DELETE	/api/products/:id	Delete product (admin)
POST	/api/orders	Create order
GET	/api/orders/my	Get current user's orders
GET	/api/orders	Get all orders (admin)
PUT	/api/orders/:id/status	Update order status (admin)
GET	/api/wishlist	Get wishlist
POST	/api/wishlist	Toggle wishlist item

10. User Workflow

1. Register → Login → Browse Products → View Details → Add to Cart → Checkout → Place Order → View Orders
2. Admin Login → Dashboard → Add/Delete Products → View Orders → Update Order Status

11. Security

- Passwords are hashed using bcryptjs before storage.
- JWT tokens are used for authenticated requests.
- Admin-only operations are protected by authorization middleware.
- Environment variables are used for database and JWT configuration.

12. Testing

Test Case	Expected Result	Status
User registration	Account created successfully	Pass
User login	JWT token returned	Pass
Product listing	Products displayed from MongoDB	Pass
Product details	Selected product displayed	Pass
Cart operations	Quantity/total updated correctly	Pass
Checkout	Order created in database	Pass
Order history	User's orders displayed	Pass
Admin product operation	Authorized admin can manage products	Pass
Order status update	Admin can update order status	Pass

13. Results

The completed application provides an end-to-end e-commerce workflow. The React frontend communicates successfully with the Node.js/Express backend, while MongoDB stores application data. Authentication, product browsing, cart, checkout, orders, wishlist and administrative operations are implemented as integrated modules.

14. Limitations

- Payment processing is not connected to a real payment gateway.
- Product images are stored as image URLs rather than uploaded files.
- Advanced recommendation and analytics features are not included.
- Production deployment and cloud database configuration are outside the academic scope.

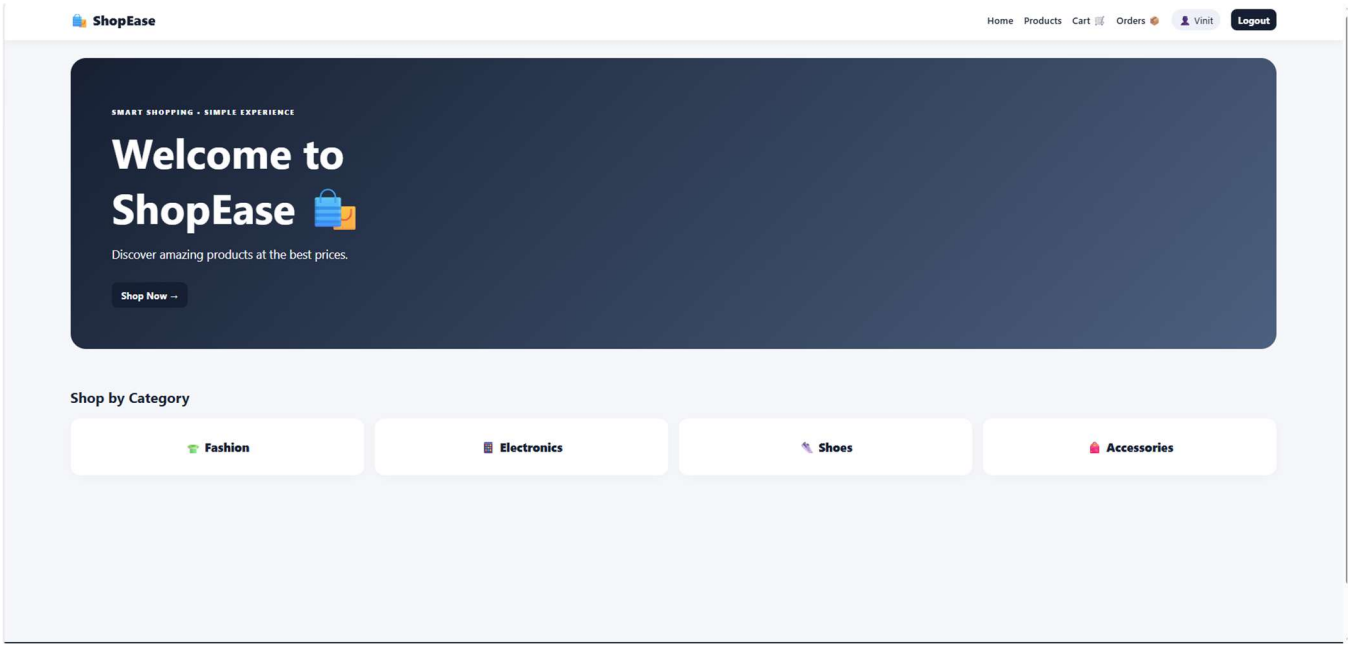
15. Future Scope

- Integrate Razorpay/Stripe or another payment gateway.
- Add product image upload and cloud storage.
- Add product reviews and ratings.
- Add advanced search, filtering and recommendation features.
- Deploy frontend, backend and database to cloud services

Application Screenshots

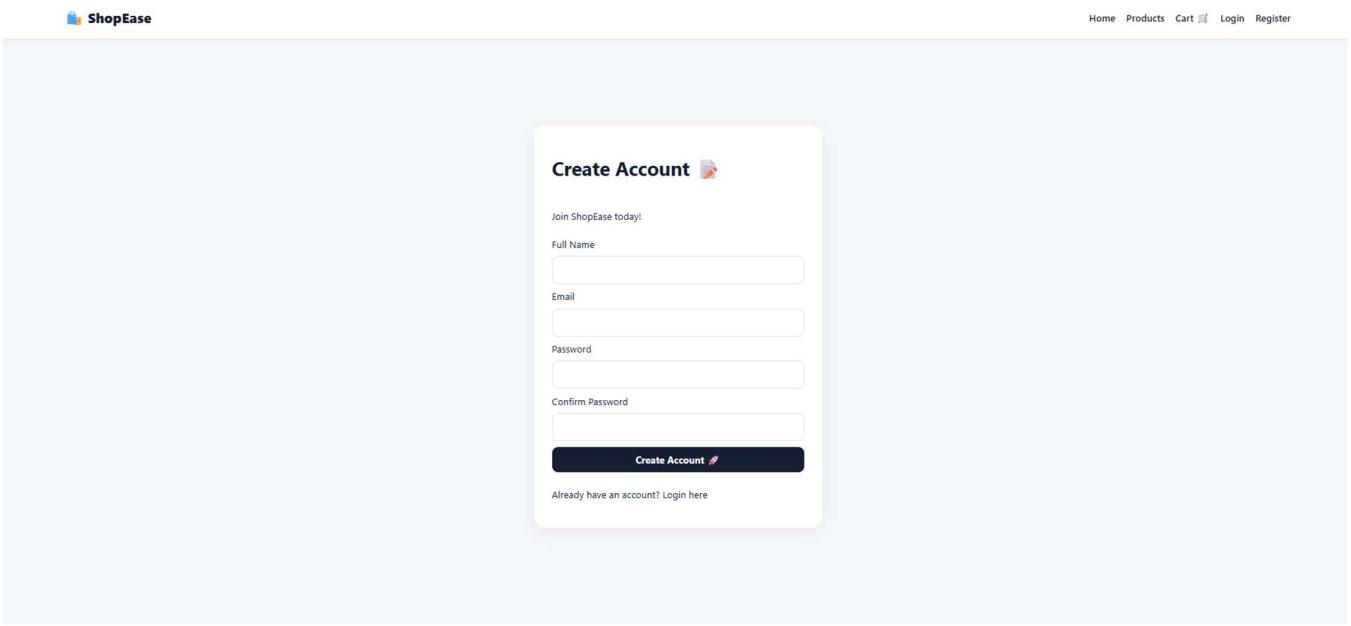
Replace the placeholders below with screenshots captured from the running ShopEase application. The captions are already prepared for the final submission.

Figure 1: Home Page



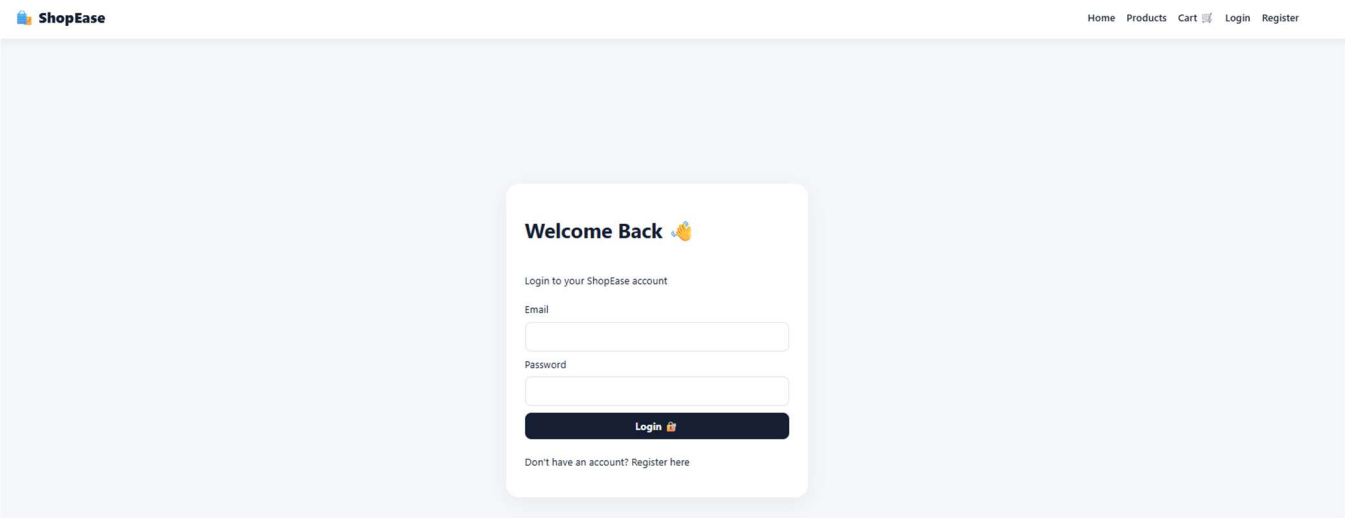
ShopEase home page showing the navigation bar and main content.

Figure 2: Register Page



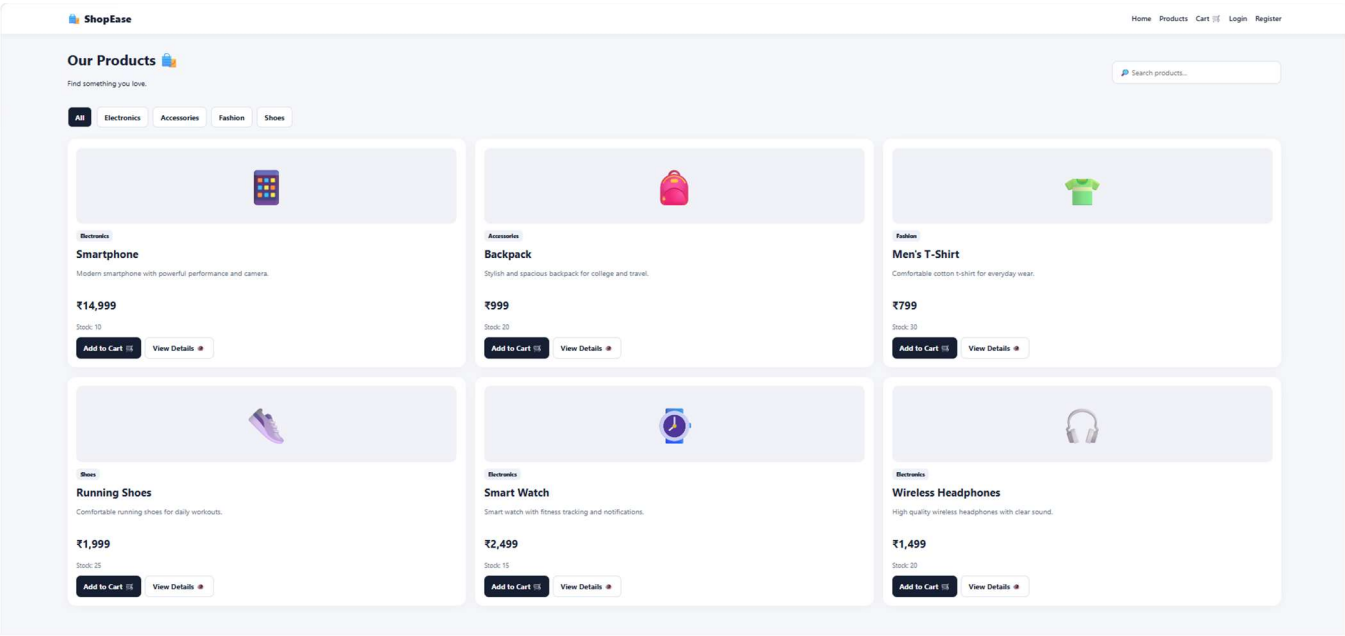
User registration form.

Figure 3: Login Page



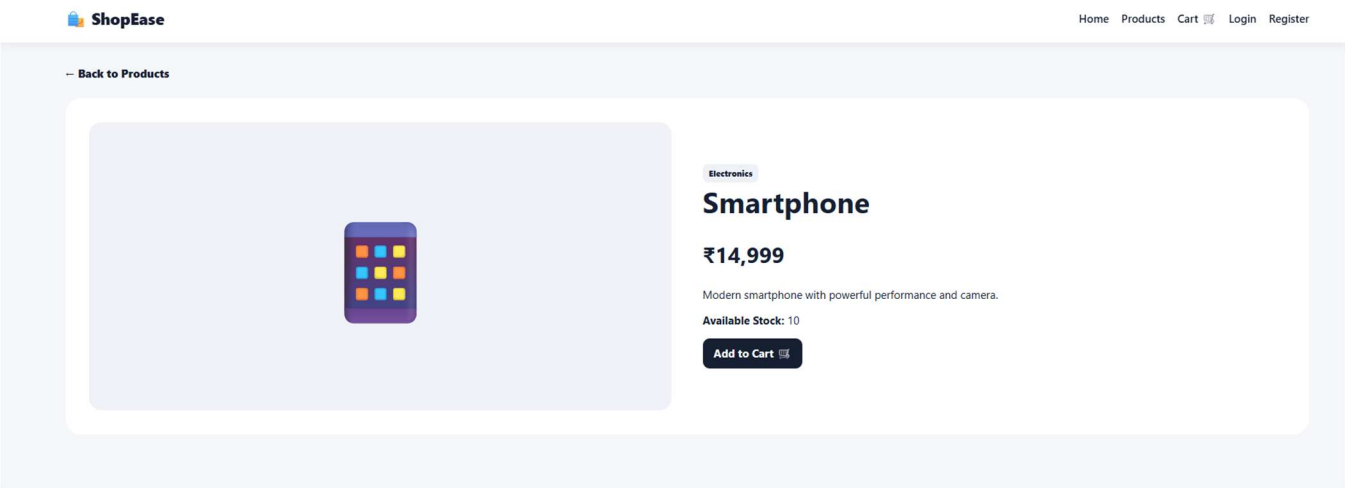
User login and authentication interface.

Figure 4: Products Page



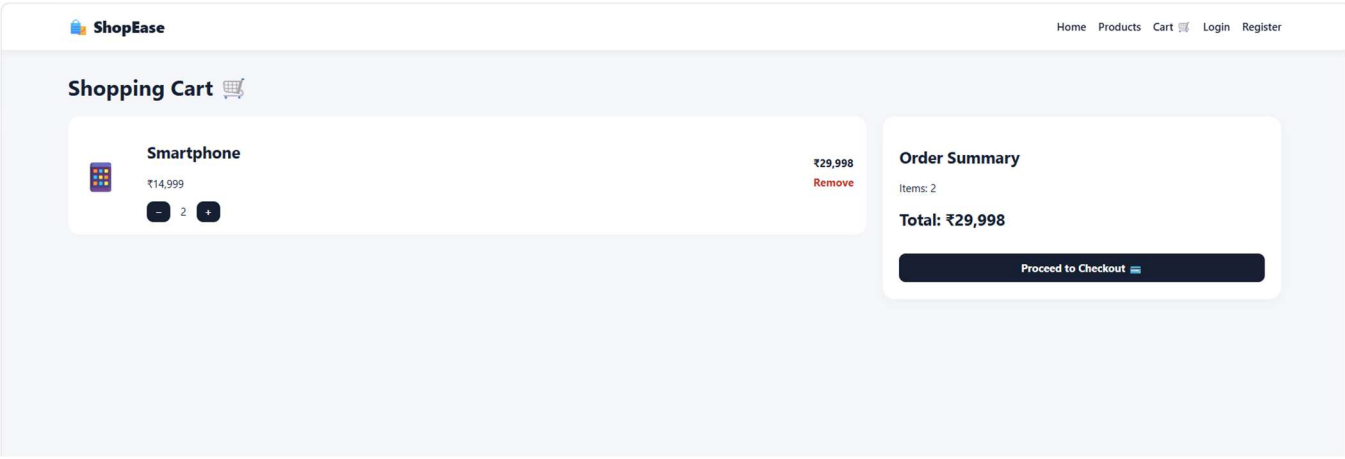
Product catalogue with product cards, categories and prices.

Figure 5: Product Details



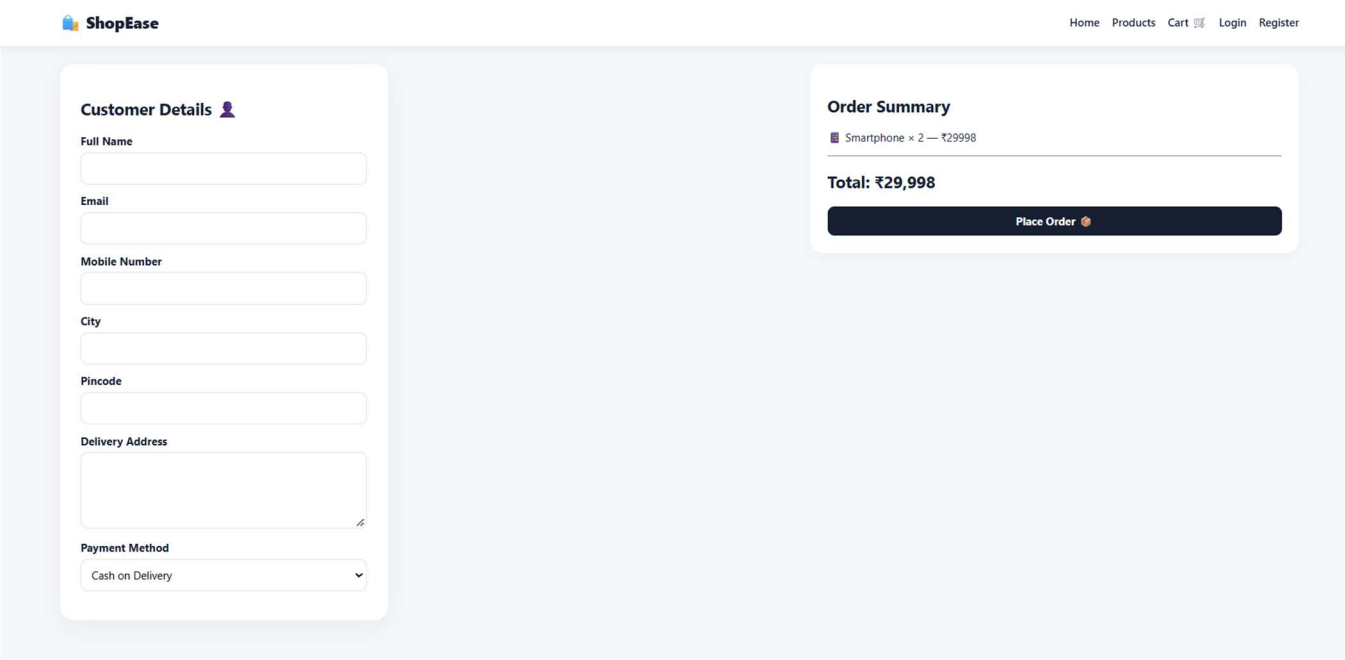
Selected product details with Add to Cart option.

Figure 6: Shopping Cart



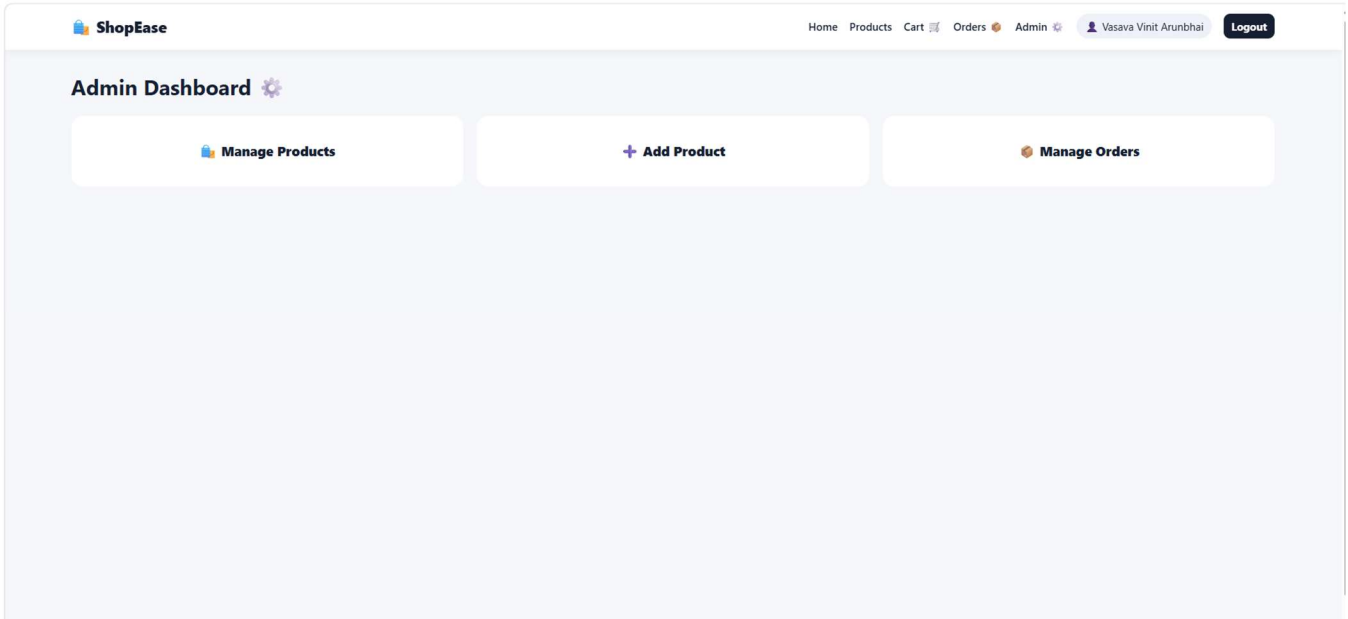
Selected products, quantities and total amount.

Figure 7: Checkout Page



Customer/delivery details and order summary.

Figure 9: Admin Dashboard



Administrator dashboard.

16. Conclusion

ShopEase successfully demonstrates the development of a full-stack e-commerce web application using modern web technologies. The project integrates React.js, Node.js, Express.js and MongoDB with authentication, REST APIs and role-based administration. It provides a practical foundation that can be extended with online payments, analytics, reviews and cloud deployment.

Filename: ShopEase_Project_Report_With_Screenshot_Places.docx
Directory: C:\Users\VINIT\OneDrive\ドキュメント
Template: C:\Users\VINIT\AppData\Roaming\Microsoft\Templates\Normal.dotm
Title:
Subject:
Author: python-docx
Keywords:
Comments: generated by python-docx
Creation Date: 24-12-2013 04:45:00
Change Number: 1
Last Saved On: 24-12-2013 04:45:00
Last Saved By:
Total Editing Time: 21 Minutes
Last Printed On: 07-10-2026 08:11:00
As of Last Complete Printing
Number of Pages: 9
Number of Words: 1,117 (approx.)
Number of Characters: 6,372 (approx.)